Java Date and Time

Aim :

To write a Java program to find the day of the week for a given date using date and time handling concepts

Algorithm :

Step 1 Start the program

Step 2 Read the month day and year

Step 3 Create a LocalDate object with the given values

Step 4 Get the day of the week from the LocalDate object

Step 5 Display the day of the week

Step 6 Stop the program

Program :

```
import java.time.LocalDate;


public class Main {
   public static void main(String[] args) {
      int month = 8;
      int day = 5;
      int year = 2015;
      LocalDate date = LocalDate.of(year, month, day);
      System.out.println("Day " + date.getDayOfWeek());
   }
}
```

Sample Output :

Day WEDNESDAY

Number of Days Between Two Dates

Aim :

To write a Java program to find the number of days between two given dates

Algorithm :

Step 1 Start the program

Step 2 Store the two given dates as strings

Step 3 Convert both strings into LocalDate objects

Step 4 Find the number of days between the two dates

Step 5 Convert the result into a positive value

Step 6 Display the number of days

Step 7 Stop the program

Program :

```
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

public class Main {
   public static void main(String[] args) {
      String dateOne = "2019-06-29";
      String dateTwo = "2019-06-30";
      long days = Math.abs(ChronoUnit.DAYS.between(LocalDate.parse(dateOne),
LocalDate.parse(dateTwo)));
      System.out.println("Days " + days);
   }
}
```

Sample Output :

Days 1

Day of the Year

Aim :

To write a Java program to find the day number of the year for a given date

Algorithm :

Step 1 Start the program

Step 2 Store the date as a string

Step 3 Convert the string into a LocalDate object

Step 4 Use the day of year method to find the required number

Step 5 Display the day number

Step 6 Stop the program

Program :

```
import java.time.LocalDate;

public class Main {
   public static void main(String[] args) {
      String date = "2019-02-10";
      int dayNumber = LocalDate.parse(date).getDayOfYear();
      System.out.println("Day Number " + dayNumber);
   }
}
```

Sample Output :

Day Number 41

Day of the Week

Aim :

To write a Java program to find the day of the week for a given day month and year

Algorithm :

Step 1 Start the program

Step 2 Read day month and year values

Step 3 Create a LocalDate object

Step 4 Find the day of the week

Step 5 Convert the day name into title case

Step 6 Display the day name

Step 7 Stop the program

Program :

```
import java.time.LocalDate;
import java.time.format.TextStyle;
import java.util.Locale;

public class Main {
   public static void main(String[] args) {
      int day = 31;
      int month = 8;
      int year = 2019;
      String result = LocalDate.of(year, month,
day).getDayOfWeek().getDisplayName(TextStyle.FULL, Locale.ENGLISH);
      System.out.println("Day " + result);
   }
```

}

Sample Output :

Day Saturday

Java Priority Queue

Aim :

To write a Java program to process students using PriorityQueue and collection processing

Algorithm :

Step 1 Start the program

Step 2 Create a Student class with id name and cgpa

Step 3 Create a PriorityQueue with a custom order

Step 4 Give higher priority to higher cgpa

Step 5 If cgpa values are equal give priority to name in alphabetical order

Step 6 If names are equal give priority to smaller id

Step 7 Process enter and served events

Step 8 Display the remaining students in priority order

Step 9 Stop the program

Program :

```
import java.util.Comparator;
import java.util.PriorityQueue;

class Student {
   int id;
   String name;
   double cgpa;

   Student(int id, String name, double cgpa) {
      this.id = id;
```

```
        this.name = name;
        this.cgpa = cgpa;
    }
}

public class Main {
    public static void main(String[] args) {
        PriorityQueue<Student> queue = new PriorityQueue<>(
                Comparator.comparingDouble((Student s) -> s.cgpa).reversed()
                        .thenComparing(s -> s.name)
                        .thenComparingInt(s -> s.id)
        );
        queue.add(new Student(50, "John", 3.75));
        queue.add(new Student(24, "Mark", 3.8));
        queue.add(new Student(35, "Shafa", 3.7));
        queue.poll();
        queue.poll();
        queue.add(new Student(36, "Dan", 3.95));
        queue.add(new Student(42, "Ashley", 3.9));
        queue.add(new Student(46, "Maria", 3.6));
        while (!queue.isEmpty()) {
            System.out.println(queue.poll().name);
        }
    }
}
```

Sample Output :

Dan

Ashley

Shafa

Maria

Java ArrayList

Aim :

To write a Java program to access elements from nested ArrayList collections

Algorithm :

Step 1 Start the program

Step 2 Create a nested ArrayList of integers

Step 3 Store the query row and column values

Step 4 For each query check whether the row and column exist

Step 5 If the element exists display the value

Step 6 Otherwise display error

Step 7 Stop the program

Program :

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Main {
  public static void main(String[] args) {
    List<List<Integer>> numbers = new ArrayList<>();
    numbers.add(Arrays.asList(41, 77, 74, 22, 44));
    numbers.add(Arrays.asList(12));
    numbers.add(Arrays.asList(37, 34, 36, 52));
    int[][] queries = {{1, 3}, {3, 5}, {3, 1}};
    for (int[] query : queries) {
      int row = query[0] - 1;
```

```
        int column = query[1] - 1;
        if (row >= 0 && row < numbers.size() && column >= 0 && column <
numbers.get(row).size()) {
            System.out.println(numbers.get(row).get(column));
        } else {
            System.out.println("ERROR");
        }
      }
   }
}
```

Sample Output :

74

ERROR

37

Largest Number

Aim :

To write a Java program to form the largest possible number from a given array using a custom comparator

Algorithm :

Step 1 Start the program

Step 2 Create an integer array nums

Step 3 Convert every number into a string

Step 4 Sort the strings using a custom comparator

Step 5 Compare two possible joined values for every pair

Step 6 Join the sorted strings to form the largest number

Step 7 Display the largest number

Step 8 Stop the program

Program :

```
import java.util.Arrays;
import java.util.stream.Collectors;

public class Main {
   public static void main(String[] args) {
      int[] nums = {3, 30, 34, 5, 9};
      String result = Arrays.stream(nums)
            .mapToObj(String::valueOf)
            .sorted((a, b) -> (b + a).compareTo(a + b))
            .collect(Collectors.joining());
      if (result.charAt(0) == '0') {
```

```
            result = "0";
        }
        System.out.println("Largest Number " + result);
    }
}
```

Sample Output :

Largest Number 9534330

Java Comparator

Aim :

To write a Java program to sort player records using a custom comparator

Algorithm :

Step 1 Start the program

Step 2 Create a Player class with name and score

Step 3 Store player records in an array

Step 4 Sort players by score in descending order

Step 5 If scores are equal sort by name in alphabetical order

Step 6 Display the sorted players

Step 7 Stop the program

Program :

```
import java.util.Arrays;
import java.util.Comparator;

class Player {
   String name;
   int score;

   Player(String name, int score) {
      this.name = name;
      this.score = score;
   }
}
```

```
public class Main {
   public static void main(String[] args) {
      Player[] players = {
            new Player("amy", 100),
            new Player("david", 100),
            new Player("heraldo", 50),
            new Player("aakansha", 75),
            new Player("aleksa", 150)
      };
      Arrays.sort(players, Comparator.comparingInt((Player p) ->
p.score).reversed().thenComparing(p -> p.name));
      for (Player player : players) {
         System.out.println(player.name + " " + player.score);
      }
   }
}
```

Sample Output :

```
aleksa 150
amy 100
david 100
aakansha 75
heraldo 50
```

Aim :

To write a Java program to sort student records using custom comparator and sorting logic

Algorithm :

Step 1 Start the program

Step 2 Create a Student class with roll number name and marks

Step 3 Store student records in a list

Step 4 Sort students by marks in descending order

Step 5 If marks are equal sort by name in alphabetical order

Step 6 Display the sorted students

Step 7 Stop the program

Program :

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

class Student {
  int rollNumber;
  String name;
  int marks;

  Student(int rollNumber, String name, int marks) {
    this.rollNumber = rollNumber;
    this.name = name;
    this.marks = marks;
  }
```

```
}


public class Main {
   public static void main(String[] args) {
      List<Student> students = new ArrayList<>();
      students.add(new Student(101, "Ravi", 88));
      students.add(new Student(102, "Anu", 95));
      students.add(new Student(103, "Kiran", 88));
      students.add(new Student(104, "Meena", 92));
      students.sort(Comparator.comparingInt((Student s) ->
s.marks).reversed().thenComparing(s -> s.name));
      for (Student student : students) {
         System.out.println(student.rollNumber + " " + student.name + " " +
student.marks);
      }
   }
}
```

Sample Output :

102 Anu 95

104 Meena 92

103 Kiran 88

101 Ravi 88

Sort the People

Aim :

To write a Java program to sort people by height in descending order

Algorithm :

Step 1 Start the program

Step 2 Create an array of names

Step 3 Create an array of heights

Step 4 Sort indexes based on height in descending order

Step 5 Use the sorted indexes to arrange names

Step 6 Display the sorted names

Step 7 Stop the program

Program :

```
import java.util.Arrays;
import java.util.stream.Collectors;
import java.util.stream.IntStream;

public class Main {
   public static void main(String[] args) {
      String[] names = {"Mary", "John", "Emma"};
      int[] heights = {180, 165, 170};
      String result = IntStream.range(0, names.length)
            .boxed()
            .sorted((a, b) -> Integer.compare(heights[b], heights[a]))
            .map(i -> names[i])
            .collect(Collectors.joining(" "));
```

```
        System.out.println("Sorted People " + result);
    }
}
```

Sample Output :

Sorted People Mary Emma John